

EcoGrid: Sistema de Gestión de Red Eléctrica Inteligente

Emanuel Aristov

Ignacio Silva

Universidad Católica del Uruguay

Algoritmos y Estructuras de Datos

Mayo 2026

Introducción

EcoGrid es un sistema desarrollado en Java cuyo objetivo es gestionar una red eléctrica inteligente. El sistema permite registrar nodos de energía y consumidores, y en base a la demanda que tiene cada consumidor, analiza qué nodos son capaces de suplir esa demanda. Cada asignación queda registrada como una transacción dentro del sistema.

El proyecto fue desarrollado en el marco de la materia Algoritmos y Estructuras de Datos, con el foco puesto en implementar desde cero las estructuras de datos necesarias para el funcionamiento del sistema, sin utilizar las colecciones estándar de Java. Este informe detalla el análisis del problema, las decisiones de diseño, las estructuras de datos implementadas con su complejidad algorítmica, y las clases principales que componen la solución.

Análisis del Problema

Una red eléctrica inteligente es un sistema que necesita gestionar de forma dinámica la producción y el consumo de energía. En este contexto, el problema central es cómo asignar eficientemente la energía disponible en los nodos productores a los consumidores que la demandan, respetando restricciones de capacidad y prioridad.

Los principales requerimientos identificados son los siguientes:

- Registrar nodos de energía con su tipo de fuente, capacidad máxima y carga actual disponible.
- Registrar consumidores con su identificador, nombre, nivel de prioridad y demanda de energía requerida.
- Procesar solicitudes de carga: dado un consumidor, encontrar el nodo o conjunto de nodos capaces de suplir su demanda.
- Registrar cada operación de asignación como una transacción con historial persistente.
- Proveer una interfaz que unifique el acceso a todas las operaciones del sistema.

El desafío principal no es solo modelar las entidades del dominio, sino elegir las estructuras de datos correctas para que las operaciones de mayor frecuencia (inserción de solicitudes, consulta de nodos disponibles y registro de transacciones) se realicen de forma eficiente.

Marco Teórico

Para resolver los requerimientos del sistema se implementaron tres estructuras de datos genéricas desde cero. A continuación se describen cada una y se justifica su elección en el contexto del proyecto.

Lista Enlazada

Una lista enlazada es una estructura lineal donde cada nodo contiene un dato y una referencia al nodo siguiente. A diferencia de un arreglo, no requiere memoria contigua y permite inserciones y eliminaciones en $O(1)$ en el frente. Es la base sobre la cual se construyeron las demás estructuras del sistema.

Cola

Una cola es una estructura FIFO (First In, First Out): el primer elemento en entrar es el primero en salir. Las operaciones de enqueue y dequeue son $O(1)$ con implementación basada en lista enlazada, lo que la hace eficiente para el procesamiento ordenado de solicitudes.

Pila

Una pila es una estructura LIFO (Last In, First Out): el último elemento en entrar es el primero en salir. Las operaciones push y pop son $O(1)$, lo que la hace ideal para acceso inmediato al elemento más reciente.

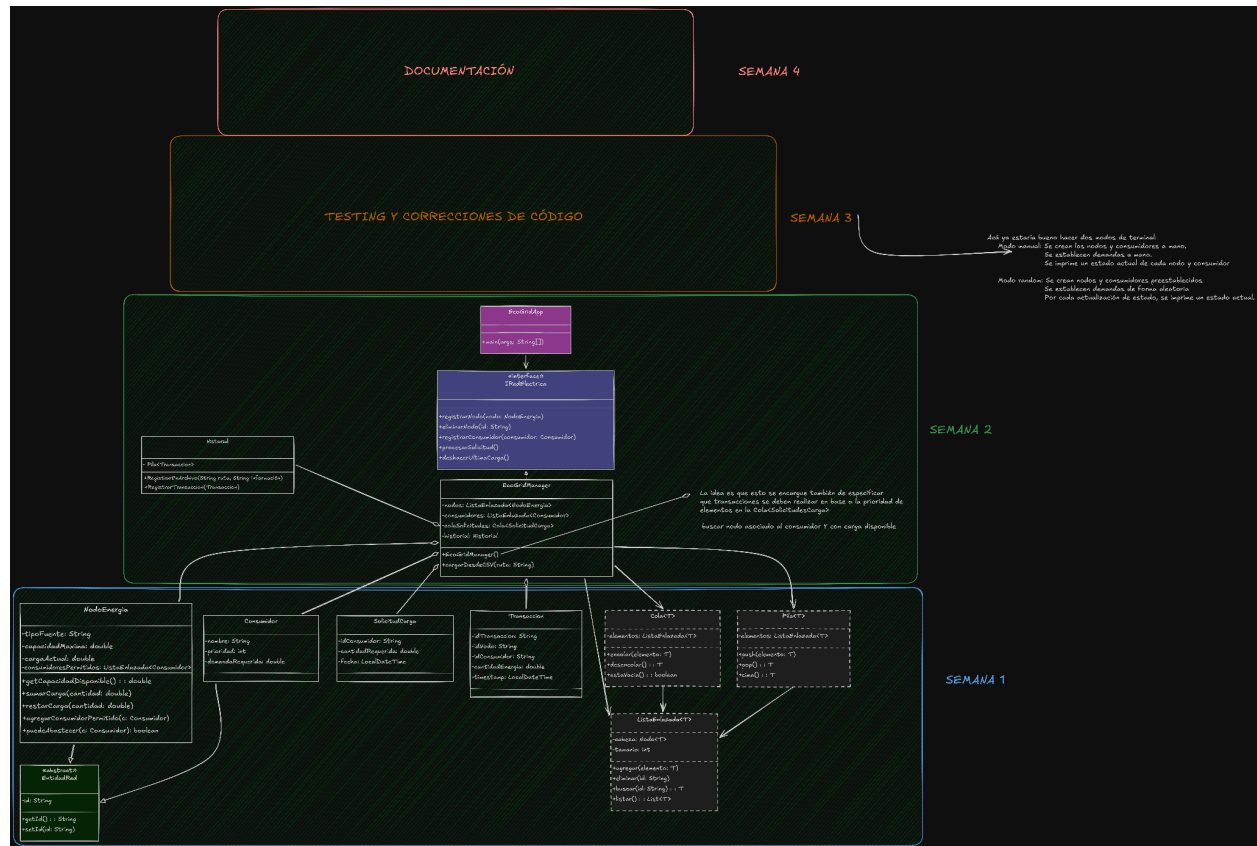
Planteo de la Solución

La solución se diseñó siguiendo principios de orientación a objetos: cada entidad del dominio se modela como una clase, las responsabilidades están delimitadas y el acceso al sistema se unifica a través de una interfaz. A continuación se presentan las decisiones de diseño más relevantes.

Decisiones de Diseño

- Las estructuras de datos se implementaron desde cero, sin utilizar ArrayList, LinkedList ni ninguna otra colección de la biblioteca estándar de Java, para cumplir con el objetivo pedagógico de la materia.
- Las estructuras son genéricas (utilizan el tipo $\langle T \rangle$), lo que permite reutilizarlas con cualquier tipo de dato sin duplicar código.
- Se definió la interfaz IRedElectrica como fachada del sistema, ocultando los detalles internos y exponiendo únicamente las operaciones necesarias para el cliente.
- EcoGridManager actúa como clase orquestadora: centraliza la lógica de negocio y coordina las estructuras de datos internas.

Diagrama UML



(<https://excalidraw.com/#room=2e8904f1dff2668d7f66,F5esEFP8JcJ4T1EH3Hk60A>)

Implementación

A continuación se describen las clases principales del sistema y su rol dentro de la solución. No se reproduce código fuente en este apartado; el detalle de implementación puede consultarse en el repositorio del proyecto.

Estructuras de Datos Genéricas

ListaEnlazada<T>

Implementa una lista simplemente enlazada con nodos internos. Provee operaciones de inserción al frente y al final, eliminación, búsqueda y recorrido. Es la estructura base del sistema y sobre ella se construyeron Cola y Pila.

Cola<T>

Implementa semántica FIFO mediante una lista enlazada subyacente. El método enqueue() agrega al final y dequeue() extrae del frente, ambos en $O(1)$. Se utiliza para gestionar la cola de solicitudes de carga pendientes de procesamiento.

Pila<T>

Implementa semántica LIFO. push() agrega al frente y pop() extrae del frente, ambos en $O(1)$. Se utiliza para mantener el historial de transacciones, donde la operación más reciente siempre está disponible en el tope.

Clases del Dominio

NodoEnergia

Representa una fuente de energía dentro de la red. Almacena un identificador único, el tipo de fuente (solar, eólica, entre otras), la capacidad máxima y la carga actual disponible para asignación.

Consumidor

Representa un punto de consumo de energía. Tiene un identificador, nombre, nivel de prioridad y la demanda de energía que requiere. El nivel de prioridad determina el orden de atención cuando los recursos son limitados.

SolicitudCarga

Encapsula el pedido de energía de un consumidor. Se genera cuando un consumidor solicita carga y se almacena en la cola de solicitudes pendientes hasta ser procesada por el sistema.

Transaccion

Registra el resultado de procesar una solicitud: qué consumidor fue atendido, qué nodo o nodos proveyeron la energía, la cantidad transferida y el momento de la operación. Las transacciones se persisten en el archivo historial_transacciones.txt.

EcoGridManager e IRedElectrica

EcoGridManager es la clase central del sistema. Mantiene internamente la lista de nodos, la lista de consumidores, la cola de solicitudes y la pila de transacciones. Contiene la lógica de asignación: al procesar una solicitud, recorre los nodos disponibles y asigna aquellos cuya capacidad libre alcanza para cubrir la demanda del consumidor.

IRedElectrica es la interfaz fachada que expone las operaciones públicas del sistema: registrar nodo, registrar consumidor, procesar solicitudes y consultar historial. EcoGridManager implementa esta interfaz, lo que permite que el cliente interactúe con el sistema sin conocer su implementación interna.

Conclusión

El desarrollo de EcoGrid permitió poner en práctica de forma concreta los conceptos trabajados durante la materia. Implementar las estructuras de datos desde cero fue lo que más aportó al aprendizaje del equipo, porque obligó a entender cómo funcionan internamente y cuándo conviene usar una sobre otra, en lugar de simplemente utilizar las que provee Java.

La elección de cada estructura no fue arbitraria: la cola garantiza que las solicitudes se procesen en orden de llegada, y la pila permite acceder siempre a la transacción más reciente de forma inmediata. Esas decisiones tienen un impacto directo en la eficiencia del sistema, algo que queda evidenciado al analizar las complejidades Big-O de cada operación.

El diseño con fachada e implementación de la interfaz IRedElectrica forzó a pensar en la separación de responsabilidades desde el principio, lo cual simplificó el desarrollo y facilitó las pruebas del sistema.

Como posibles mejoras a futuro, podría implementarse un algoritmo de prioridad más sofisticado para la asignación de nodos, soporte para múltiples tipos de demanda simultánea, o persistencia de datos más robusta que el archivo de texto actual.